

alex@ai-~huggi X test.ipynb circular_grid_500x wafer_round_dat x wafer.csv x WEBRAG_INGES x Python 3 (pykernel)

File Edit View Run Kernel Tabs Settings Help

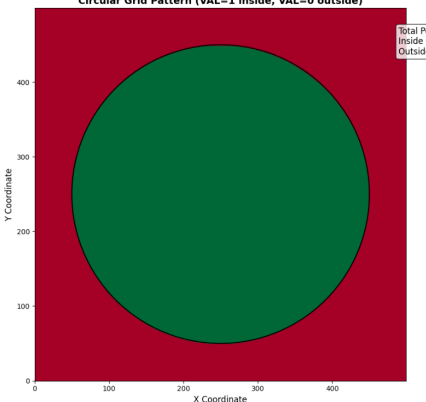
hugging_face_rag /

Name	Modif...	Size
automationRAG	16h ago	
chroma_db_data	23m ago	
data_analysis_files	16m ago	
docs	8d ago	
models	2d ago	
other_scripts	2d ago	
v939RAG	2d ago	
circular_grid_500x500...	1s ago	2.3 MB
circular_grid_heatmap...	9s ago	228.8 KB
outlier_heatmap_inter...	28m ago	4.6 MB
outlier_heatmap.png	17m ago	36.8 KB
pdf2md.py	yesterday	11.7 KB
test.ipynb	next yr	593.6 KB
three_tests_interactiv...	1h ago	4.6 MB
wafer_round_data.csv	9m ago	28.4 KB
wafer.csv	2h ago	14.9 KB
wafer.stdf	4h ago	42.7 KB
WEBRAG_INGES.py	18h ago	10.4 KB
WEBRAG_simple.html	2d ago	13.7 KB
WEBRAG_simple.py	17h ago	10.3 KB

test.ipynb

Heatmap saved to: circular_grid_heatmap.png

Circular Grid Pattern (VAL=1 inside, VAL=0 outside)



RAG Assistant

> create a csv file 500x500 grid, with inside circle, everything inside has value 1 outside 0, base line x,y, val [0 or 1]

The user wants a CSV file with: 1. 500x500 grid (250,000 points) 2. Circular pattern - value=1 inside circle, value=0 outside 3. Columns: x, y, val [0 or 1]

This is for testing the heatmap visualization with a clear circular boundary. I'll create a script that generates this data efficiently using numpy broadcasting and saves to CSV.

```
# ASSUMED: 'BIN' column contains hard bin number
# ASSUMED: 'X_COORD', 'Y_COORD' are integer die coordinates
# ASSUMED: 'WAFER_ID' and 'LOT_ID' columns exist for grouping
# VERIFY: confirm passing bin number with test engineer (often bin 1, not always)

import numpy as np
import pandas as pd

def generate_circular_grid_csv(grid_size=500, circle_radius=250, center_x=250, center_y=250):
    """Generate 500x500 grid with circular pattern (1 inside, 0 outside)."""

    print(f"Grid Size: {grid_size} x {grid_size}")
    print(f"Circular Grid Generation")
    print(f"Circle Radius: {circle_radius} x {circle_radius}")

    # Create coordinate arrays for efficient broadcasting
    x = np.arange(grid_size)
    y = np.arange(grid_size)

    # Create meshgrid (500x500 grid)
    x_grid, y_grid = np.meshgrid(x, y)
```

alex@ai-~huggi X test.ipynb test_data.csv test_data.csv Python 3 (pykernel)

File Edit View Run Kernel Tabs Settings Help

hugging_face_rag /

Name	Mo...	Size
FREQ_TEST...	30m ago	158.7 KB
FREQ_TEST...	4s ago	81.8 KB
FREQ_TEST...	6s ago	103.2 KB
FREQ_TEST...	6s ago	115.8 KB
ICC_TEST...	9s ago	92 KB
ICC_TEST...	7s ago	87.9 KB
ICC_TEST...	8s ago	81.3 KB
ICC_TEST...	8s ago	87.9 KB
ICC_TEST...	8s ago	81.3 KB
ICC_TEST...	8s ago	116.9 KB
ICC_TEST...	7s ago	80.4 KB
ICC_TEST...	9s ago	92 KB
ICC_TEST...	7s ago	108.7 KB
parametr...	11m ago	196.4 KB
pdf2md.py	yesterday	11.7 KB
test_data...	41m ago	14.9 KB
test.ipynb	10s ago	1.5 MB
VCC_TEST...	11s ago	91.5 KB
VCC_TEST...	10s ago	91 KB
VCC_TEST...	10s ago	85.4 KB
VCC_TEST...	10s ago	91 KB
VCC_TEST...	11s ago	85.4 KB
VCC_TEST...	11s ago	120.2 KB
VCC_TEST...	9s ago	81.3 KB
VCC_TEST...	12s ago	91.5 KB
VCC_TEST...	6s ago	166 KB
VCC_TEST...	9s ago	109.5 KB
wafer_an...	37m ago	50.5 KB
wafer.stdf	10h ago	42.7 KB

test.ipynb

```
plot_7_parametric_by_lot(df, param)
plt.savefig(f'{param}_by_lot.png', dpi=300, bbox_inches='tight')

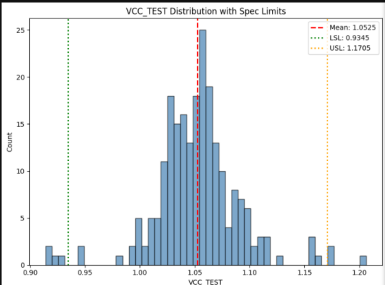
plot_8_parametric_by_site(df, param)
plt.savefig(f'{param}_boxplot_site2.png', dpi=300, bbox_inches='tight')

plot_9_parametric_by_wafer_heatmap(df, param)
plt.savefig(f'{param}_wafer_map.png', dpi=300, bbox_inches='tight')

plot_10_parametric_cpk_chart(df, param)
plt.savefig(f'{param}_cpk.png', dpi=300, bbox_inches='tight')

print("All plots saved successfully!")
```

VCC_TEST Distribution with Spec Limits



RAG Assistant

> present 10 statistical plots and names that i can use for parametric tests

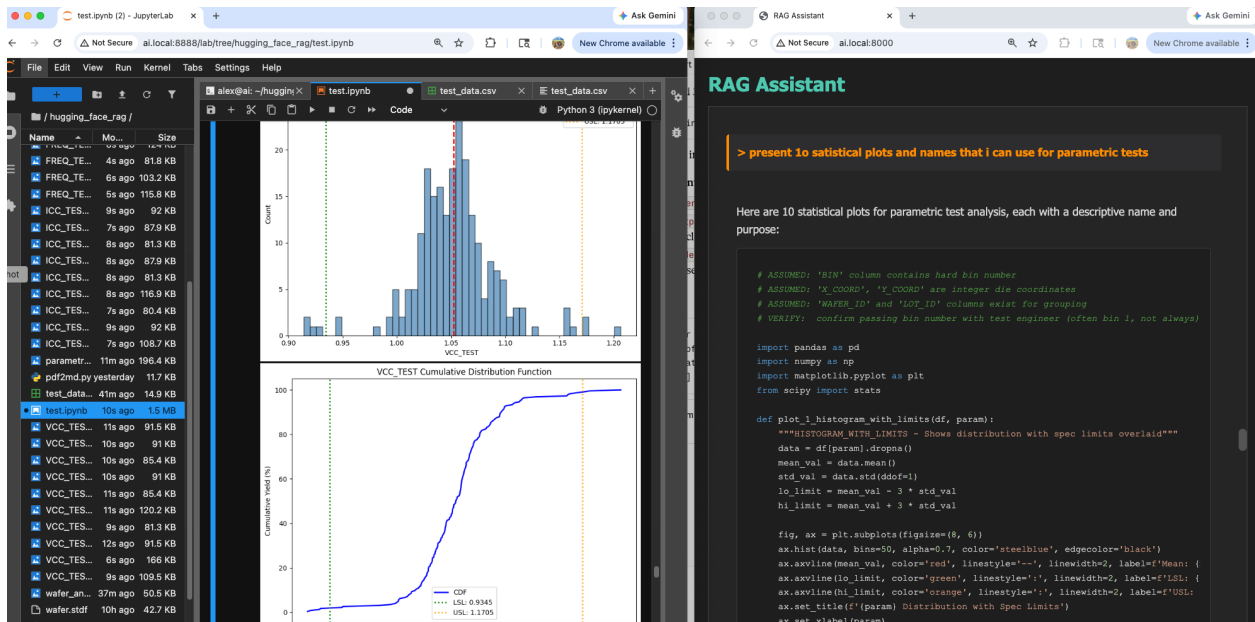
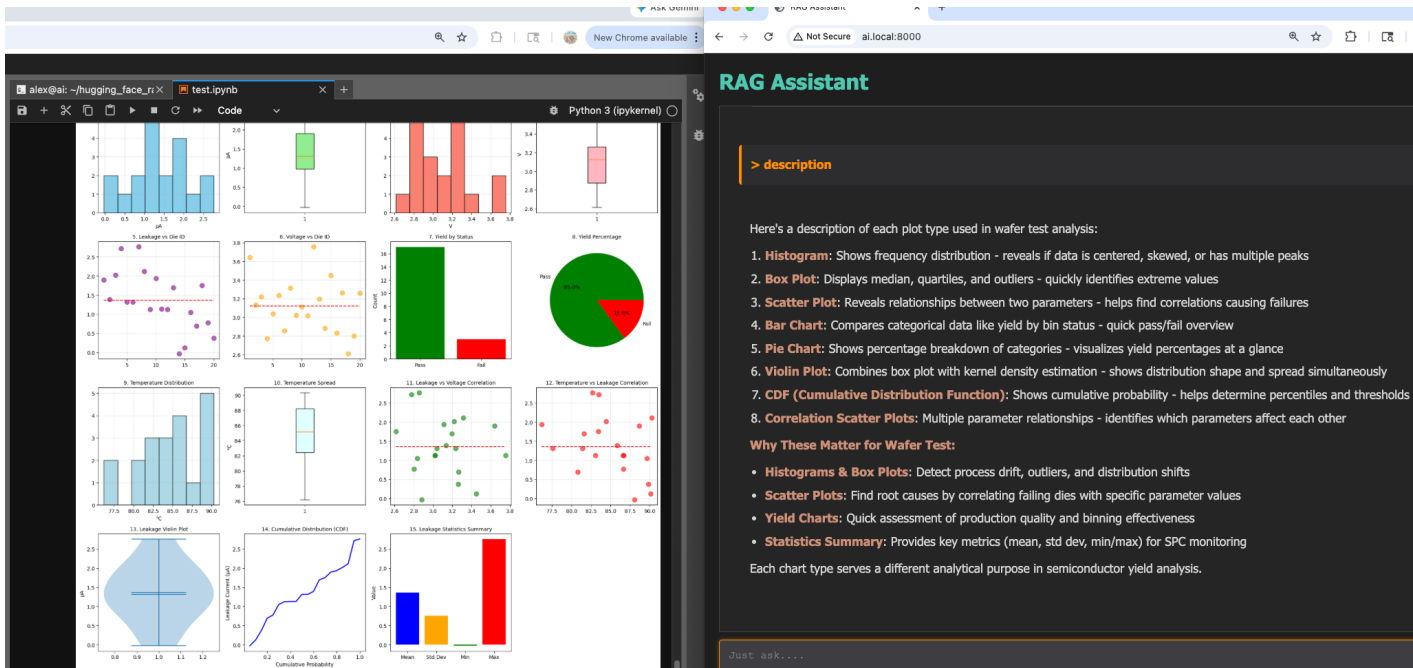
Here are 10 statistical plots for parametric test analysis, each with a descriptive name and purpose:

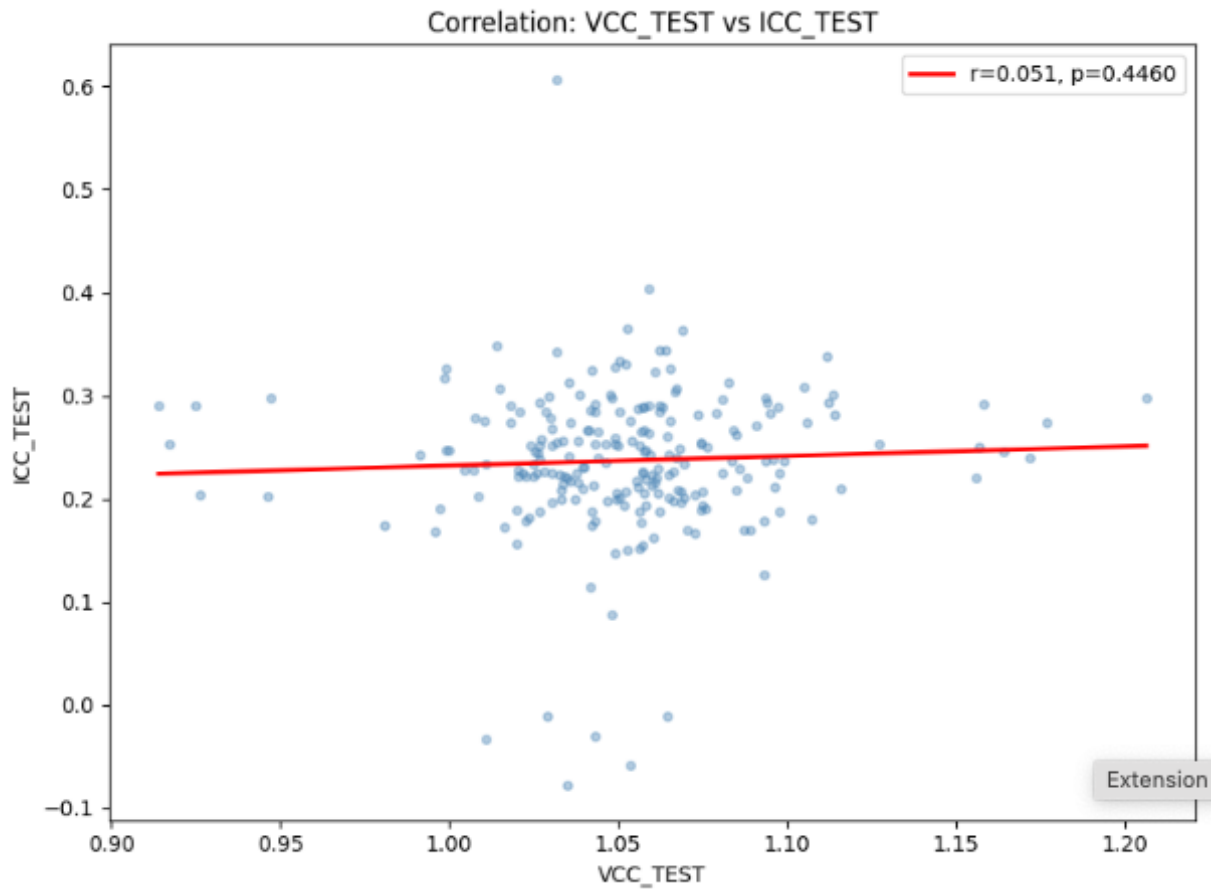
```
# ASSUMED: 'BIN' column contains hard bin number
# ASSUMED: 'X_COORD', 'Y_COORD' are integer die coordinates
# ASSUMED: 'WAFER_ID' and 'LOT_ID' columns exist for grouping
# VERIFY: confirm passing bin number with test engineer (often bin 1, not always)

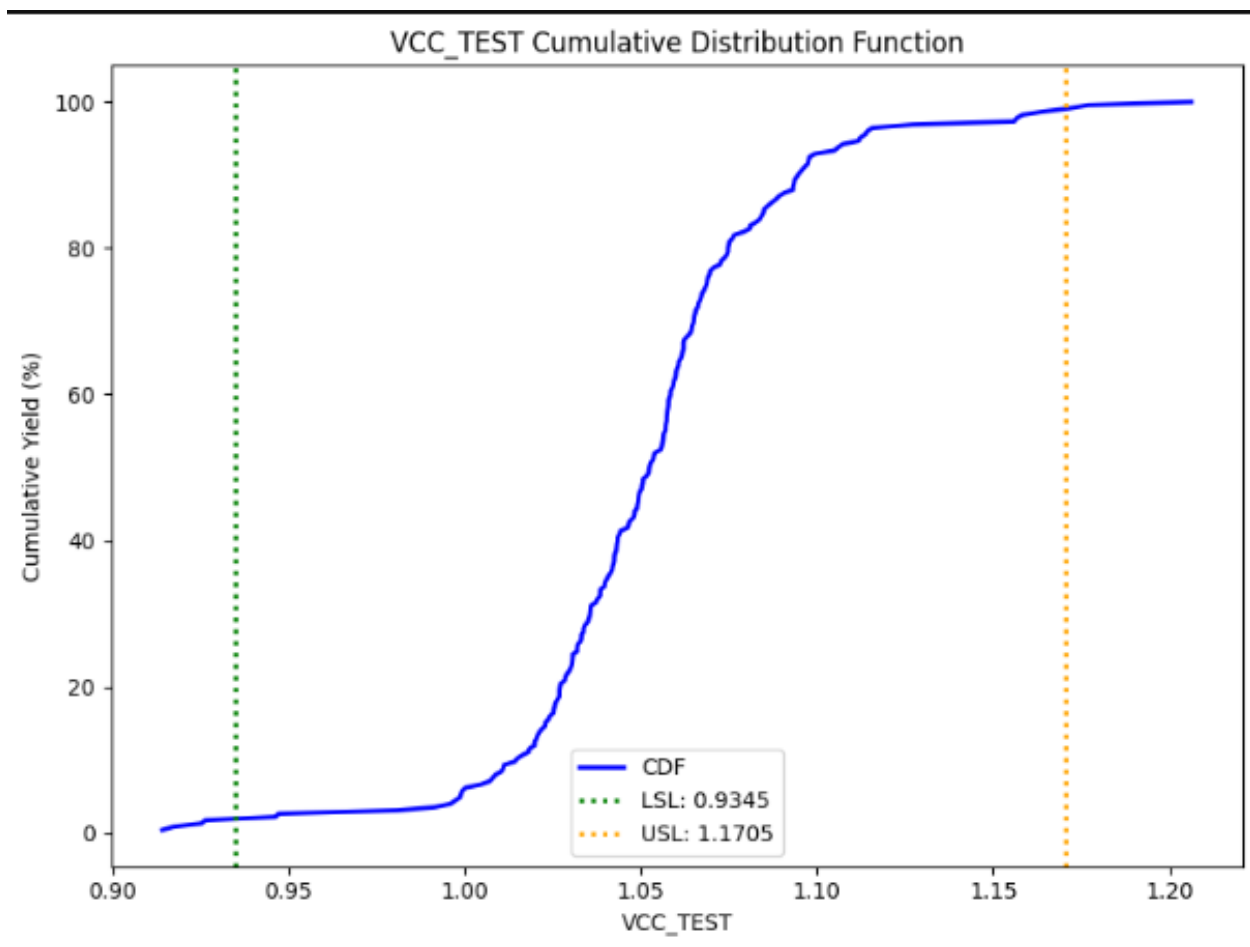
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

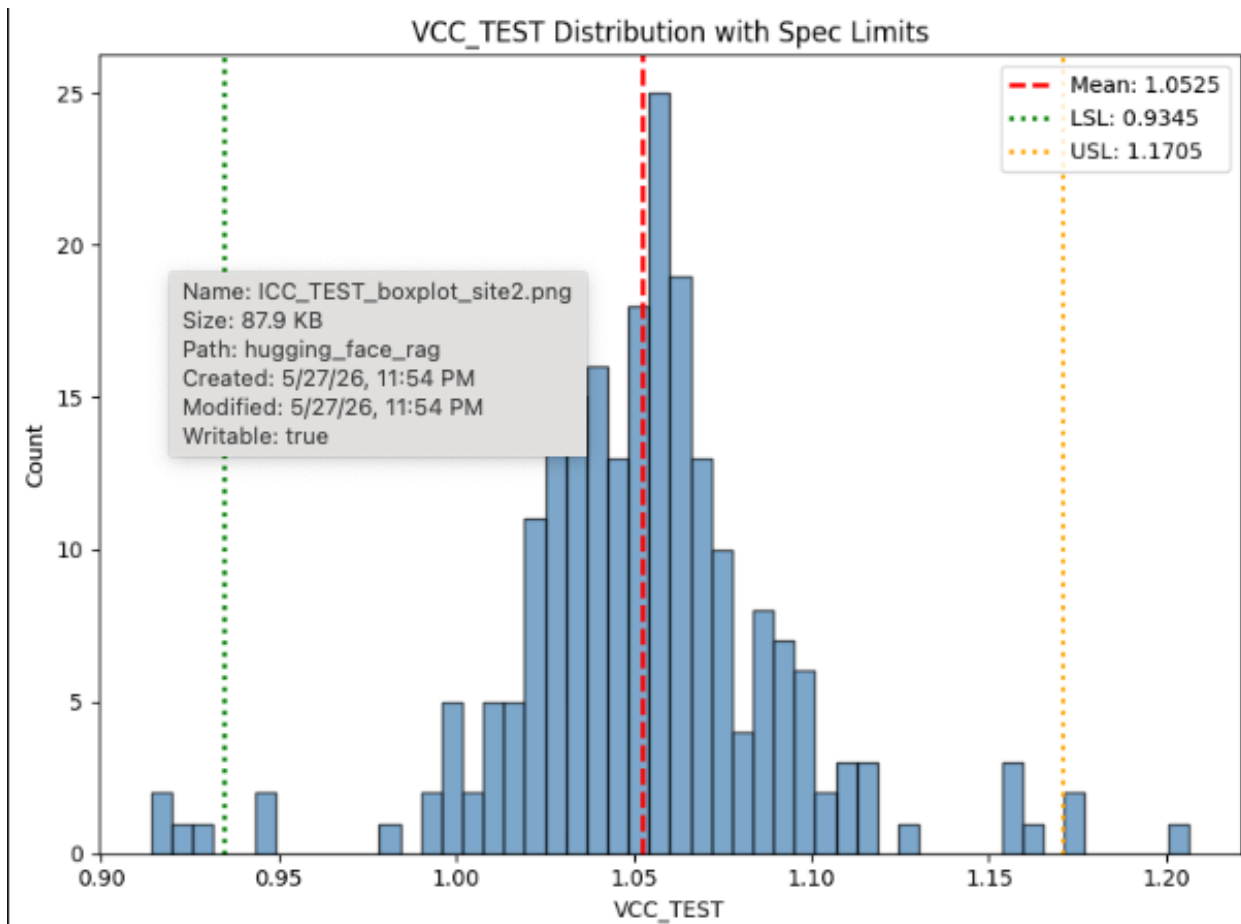
def plot_1_histogram_with_limits(df, param):
    """Histogram with Spec Limits - Shows distribution with spec limits overlaid"""
    data = df[param].dropna()
    mean_val = data.mean()
    std_val = data.std(ddof=1)
    lo_limit = mean_val - 3 * std_val
    hi_limit = mean_val + 3 * std_val

    fig, ax = plt.subplots(figsize=(8, 6))
    ax.hist(data, bins=50, alpha=0.7, color='steelblue', edgecolor='black')
    ax.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'Mean: {mean_val}')
    ax.axvline(lo_limit, color='green', linestyle='--', linewidth=2, label=f'LSL: {lo_limit}')
    ax.axvline(hi_limit, color='orange', linestyle='--', linewidth=2, label=f'USL: {hi_limit}')
    ax.set_title(f'{param} Distribution with Spec Limits')
```



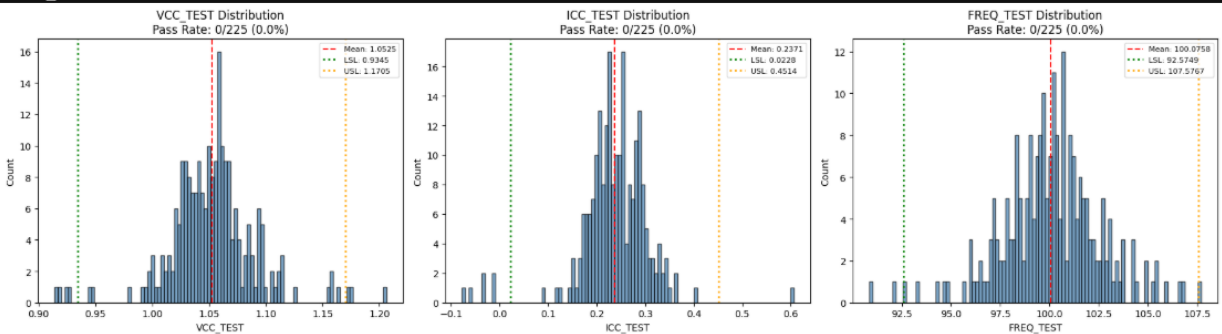






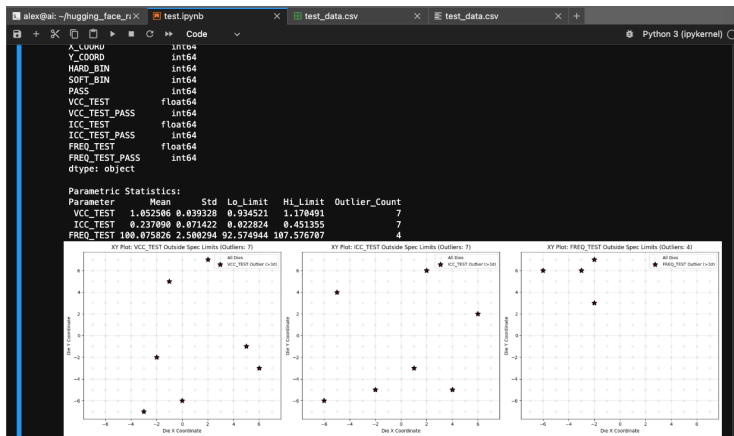
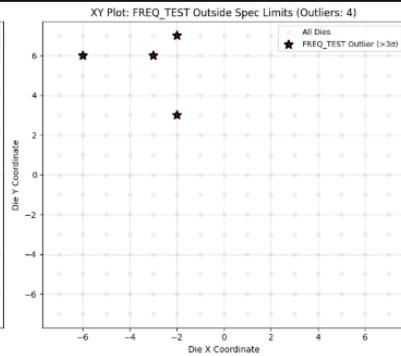
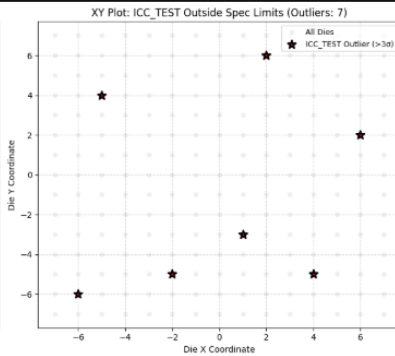
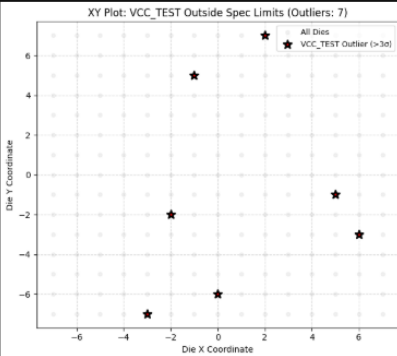
Parametric Statistics:

Parameter	Mean	Std	Min	Max	Lo_Limit	Hi_Limit	Pass_Rate
VCC_TEST	1.052506	0.039328	0.914100	1.206300	0.934521	1.170491	0.0
ICC_TEST	0.237090	0.071422	-0.077500	0.606200	0.022824	0.451355	0.0
FREQ_TEST	100.075826	2.500294	90.853203	107.705498	92.574944	107.576707	0.0



Parametric Statistics:

Parameter	Mean	Std	Lo_Limit	Hi_Limit	Outlier_Count
VCC_TEST	1.052506	0.039328	0.934521	1.170491	7
ICC_TEST	0.237090	0.071422	0.022824	0.451355	7
FREQ_TEST	100.075826	2.500294	92.574944	107.576707	4



RAG Assistant

> mark all dies that outside high limit low limit, and plot then on X Y plot. (VCC TEST)

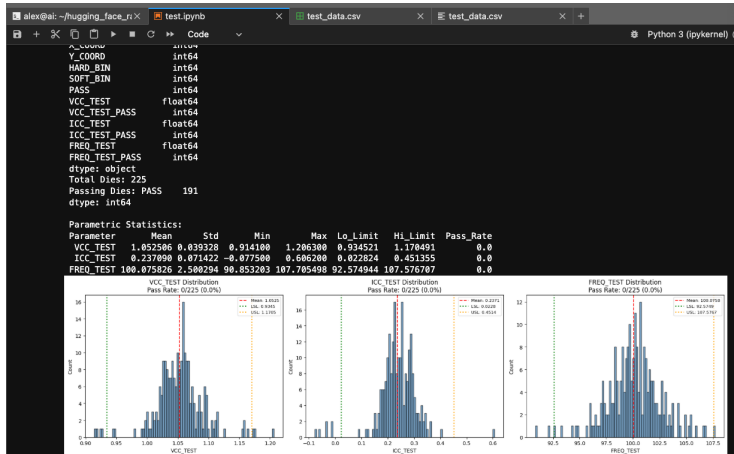
```
# ASSUMED: 'BIN' column contains hard bin number
# ASSUMED: 'X_COORD', 'Y_COORD' are integer die coordinates
# ASSUMED: 'WAFER_ID' and 'LOT_ID' columns exist for grouping
# VERIFY: confirm passing bin number with test engineer (often bin 1, not always)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# --- LOAD ---
df = pd.read_csv('test_data.csv')
print(df.head())
print(df.dtypes)

# --- CLEAN ---
params = ['VCC_TEST', 'ICC_TEST', 'FREQ_TEST']
results = df[params].astype(float).dropna()
passes = df[['PASS']].astype(int).dropna().reset_index(drop=True)

merged = pd.merge(results, passes, left_index=True, right_index=True)
merged['X_COORD'] = df.loc[merged.index, 'X_COORD'].values
merged['Y_COORD'] = df.loc[merged.index, 'Y_COORD'].values
```



[70]: # ASSUMED: 'BIN' column contains hard bin number
ASSUMED: 'X_COORD', 'Y_COORD' are integer die coordinates
ASSUMED: 'WAFER_ID' and 'LOT_ID' columns exist for grouping
VERIFY: confirm passing bin number with test engineer (often bin 1, not always)

RAG Assistant

```
# ASSUMED: 'BIN' column contains hard bin number
# ASSUMED: 'X_COORD', 'Y_COORD' are integer die coordinates
# ASSUMED: 'WAFER_ID' and 'LOT_ID' columns exist for grouping
# VERIFY: confirm passing bin number with test engineer (often bin 1, not always)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

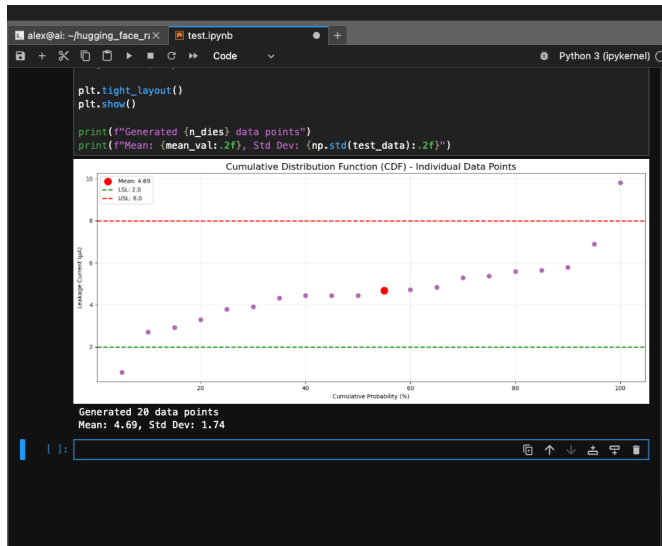
# --- LOAD ---
df = pd.read_csv('test_data.csv')
print(df.head())
print(df.dtypes)

# --- CLEAN ---
params = ['VCC_TEST', 'ICC_TEST', 'FREQ_TEST']
results = df[params].astype(float).dropna()
passes = df[['PASS']].astype(int).dropna().reset_index(drop=True)

merged = pd.merge(results, passes, left_index=True, right_index=True)
merged['X_COORD'] = df.loc[merged.index, 'X_COORD'].values
merged['Y_COORD'] = df.loc[merged.index, 'Y_COORD'].values
merged['WAFER_ID'] = df.loc[merged.index, 'WAFER_ID'].values

# --- ANALYZE ---
print(f"Total Dies: {len(df)}")
print(f"Passing Dies: {passes.sum()}")

stats_summary = {}
for param in params:
    col_data = merged[param]
    mean_val = col_data.mean()
```



RAG Assistant

> cumulative distribution

```
import matplotlib.pyplot as plt
import numpy as np

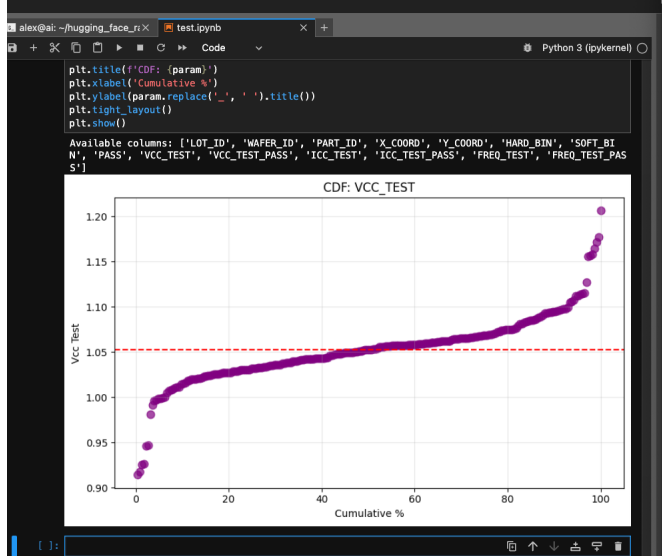
# Generate wafer test data (20 dies)
np.random.seed(42)
die_ids = range(1, 21)
n_dies = len(die_ids)

# Generate parametric data with known mean and std dev
mean_val = 5.0
std_dev = 1.2
test_data = np.random.normal(loc=mean_val, scale=std_dev, size=n_dies)

# Add some outliers to make it realistic
test_data[3] = mean_val + 4 * std_dev # Outlier above USL
test_data[7] = mean_val - 3.5 * std_dev # Near LSL

# Define specifications (LSL, USL)
ls_lsl = 2.0
usl_usl = 8.0

# Calculate Cumulative Distribution Function (CDF)
sorted_data = np.sort(test_data)
cumulative_prob = np.arange(1, len(sorted_data)+1) / len(sorted_data) * 100
```



RAG Assistant

> test_data.csv - fetch Freq test and present with cumulative distribution , show datapoints

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Load test data from CSV file
try:
    df = pd.read_csv('test_data.csv')
    print(f"Loaded {len(df)} records")
    print(f"Columns: {df.columns.tolist()}")
except FileNotFoundError:
    # Generate sample data if file doesn't exist
    np.random.seed(42)
    n_dies = 100
    df = pd.DataFrame({
        'die_id': range(1, n_dies+1),
        'freq_mhz': np.random.normal(loc=500, scale=30, size=n_dies),
        'leakage_uA': np.random.normal(loc=1.5, scale=0.8, size=n_dies)
    })
    df.to_csv('test_data.csv', index=False)
    print(f"Created sample test_data.csv with {n_dies} records")

# Select frequency parameter for analysis
param_col = 'freq_mhz' if 'freq_mhz' in df.columns else 'leakage_uA'
print(f"Analyzing: {param_col}")
```